

ECE 220 Computer Systems & Programming

Lecture 2 — Reusing Code: TRAPs and Subroutines

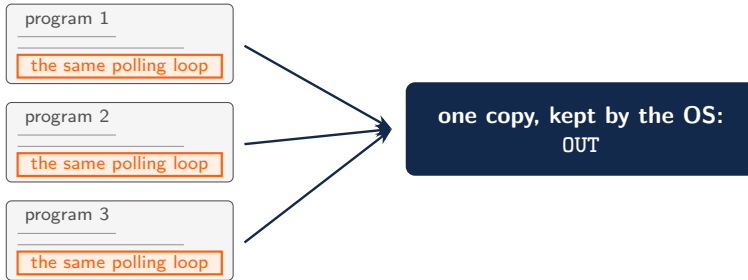
August 27, 2026

Prof. Sayan Mitra mitras@illinois.edu Office hours: TBD
Course website: courses.grainger.illinois.edu/ece220/fa2026

 **ILLINOIS**
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

The Code Reuse Problem

**Every program you will ever write prints characters.
Do we paste the polling loop into each one, forever?**



Today's Two Questions

**Reuse needs a jump *there* —
and a guaranteed way *back*.**

1. How do we *reuse* code without copying it?
2. After the borrowed code runs, how does the machine find its way *back*?

Answers today: TRAPs (system calls), subroutines (JSR/RET) — fundamental abstraction.

From Lecture 1: Echo, Two Ways

You write the I/O (Lecture 1):

```
; Lecture 1: echo by polling
.ORIG x3000
POLL1  LDI  R1, KBSR_A    ; wait for a key
        BRzp POLL1
        LDI  R0, KBDR_A    ; read it
POLL2  LDI  R1, DSR_A    ; wait for screen
        BRzp POLL2
        STI  R0, DDR_A    ; write it
        HALT
KBSR_A  .FILL xFE00
KBDR_A  .FILL xFE02
DSR_A   .FILL xFE04
DDR_A   .FILL xFE06
.END
```

The OS does it for you:

```
; Lecture 2: echo by TRAPs
.ORIG x3000
        GETC                      ; TRAP x20: R0
        <- key
        OUT                        ; TRAP x21:
        screen <- R0
        HALT                      ; TRAP x25
.END
```

- Same behavior, two instructions
- GETC = TRAP x20, OUT = TRAP x21

Where did the polling loop go? Into the operating system.

Why the OS Owns I/O

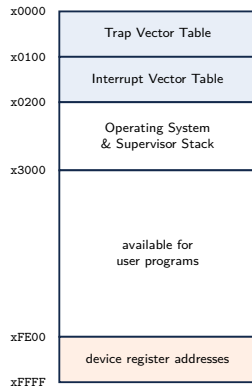
Problem with user programs doing raw I/O:

- Requires too many device details (register addresses, handshake protocols, ...)
- Security: I/O resources *shared* by many programs

Solution: make I/O part of the OS — **service routines** (**system calls**)

1. User program invokes the system call
2. OS code performs the operation
3. Control returns to the user program

In LC-3, all three steps happen through the **TRAP** mechanism.



The TRAP Instruction

TRAP	1111	0000	trapvect8
------	------	------	-----------

- `trapvect8`: an 8-bit **index** into the trap vector table (`x0000–x00FF`), zero-extended to a 16-bit address
- The table entry holds the **starting address** of the service routine
- By convention, **R0** carries the argument in and the result out

Question: how many *different* TRAP routines can the LC-3 support? $2^8 = 256$ One table entry per 8-bit vector: `x00` through `xFF`.

The TRAP Service Routines You Get

Vector	Symbol	What it does
x20	<u>GETC</u>	read one character into R0 (no echo)
x21	<u>OUT</u>	write the character in R0 to the monitor
x22	<u>PUTS</u>	write the string starting at M[R0] (one char per word)
x23	<u>IN</u>	print a prompt, read <i>and echo</i> one character into R0
x24	<u>PUTSP</u>	write the string at M[R0], two chars per word
x25	<u>HALT</u>	stop the program

The assembler accepts either spelling: GETC or TRAP x20. R0 is always the argument/result register.

PUTS vs PUTSP

```
; PUTS: one char per word; PUTSP: two per word
.ORIG x3000
    LEA R0, S1
    PUTS                ; prints abcd
    LEA R0, S2
    PUTSP               ; also prints abcd
    HALT
S1    .STRINGZ "abcd"
S2    .FILL x6261        ; 'b''a' (low byte first)
      .FILL x6463        ; 'd''c'
      .FILL x0000        ; terminator
.END
```

Memory after assembly:

```
S1:  x0061  'a'
      x0062  'b'
      x0063  'c'
      x0064  'd'
      x0000
S2:  x6261  'b','a'
      x6463  'd','c'
      x0000
```

Both print `abcd`.

`PUTSP` packs two chars per word, *low byte first* — half the memory, same output.

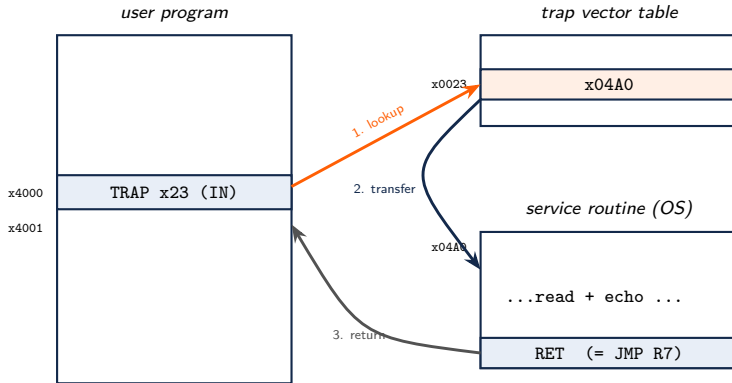
In-Class Problem: Print a String

Write a program that prints `Hello, World!` to the monitor.

- Where does the string live?
- What does `PUTS` expect in `R0`?
- Which instruction puts an *address* in a register?

```
; Print a string with PUTS
.ORIG x3000
    LEA    R0, MSG          ; R0 <- address of
                             MSG
    PUTS                               ; TRAP x22: print
    string
    HALT
MSG    .STRINGZ "Hello, World!"
.END
```

TRAP Mechanism Operation



1. **Lookup:** $R7 \leftarrow PC$; $MAR \leftarrow x0023$; $MDR \leftarrow M[x0023]$
3. **Return:** `RET` does $PC \leftarrow R7$ — execution resumes at `x4001`

2. **Transfer:** $PC \leftarrow x04A0$
TRAP saves the way home in R7.

Subroutines: Your Own Service Routines

The same call/return machinery is available to *your* code:

JSR	0100	1	PCoffset11		
JSRR	0100	0	00	BaseR	000000
RET	1100	000	111	000000	

- JSR label: $R7 \leftarrow PC$, then $PC \leftarrow PC + PCoffset11$
- JSRR BaseR: $R7 \leftarrow PC$, then $PC \leftarrow BaseR$ (reach *any* address)
- RET: $PC \leftarrow R7$ (it is literally `JMP R7`)

Demystifying R7

R7 is the return-address register.

- TRAP, JSR, and JSRR all overwrite R7 with the way home
- So: avoid using R7 to hold data — one OUT and it is gone
- If a subroutine itself calls JSR or any TRAP (even HALT's cousin PUTS), it must first save R7 — and restore it before RET

Classic bug: a subroutine prints a debug message with PUTS, then RET jumps to the middle of the OS instead of your caller.

What happens if a subroutine calls itself?

Calling a Subroutine

Lecture 1's multiply, promoted to a **subroutine**:

- **Inputs:** R0, R1
- **Output:** $R2 = R0 \times R1$
- Written once — called twice with different arguments

```
.ORIG x3000
    AND R0, R0, #0
    ADD R0, R0, #5    ; R0 = 5
    AND R1, R1, #0
    ADD R1, R1, #4    ; R1 = 4
    JSR MULT          ; R2 ← 5*4
    ST  R2, RES1
    AND R0, R0, #0
    ADD R0, R0, #6    ; R0 = 6
    AND R1, R1, #0
    ADD R1, R1, #3    ; R1 = 3
    JSR MULT          ; R2 ← 6*3
    ST  R2, RES2
    HALT
```

In-Class Problem: Write the Subroutine

Fill in `MULT` ($R2 \leftarrow R0 \times R1$, with $R1 \geq 0$):

- Which registers does it *clobber*?
- What happens if the calling program *uses* the same register?
- **Callee-save:** `ST` them on entry, `LD` them back before returning
- How do we get back to the caller?

```
; ---- MULT: R2 <- R0 * R1 (R1 >= 0) ----
MULT    ST    R3, SAVER3    ; callee-save R3
        AND   R2, R2, #0    ; product = 0
        ADD   R3, R1, #0    ; counter = R1
MLOOP   BRz   MDONE        ; counter zero?
        ADD   R2, R2, R0    ; product += R0
        ADD   R3, R3, #-1   ; count down
        BR    MLOOP
MDONE   LD    R3, SAVER3    ; restore R3
        RET                               ; PC <- R7
```

Register-Saving Conventions

- **Callee-saved** (our course convention): a subroutine saves *every* register it modifies — except the ones that carry results — and restores them before `RET`
- Mechanics today: one `ST Rk`, `SAVERk` per register on entry, matching `LD` before return
- Always save **R7** if the subroutine calls anything at all

One labeled **.FILL** slot per register, per subroutine? That cannot scale — and it breaks the moment a subroutine calls itself. Next lecture: the stack.

Summary & What's Next

Today

- **The problem:** repeated code — and user programs that should not touch devices directly
- **TRAPs** = system calls: 8-bit vector → table lookup → OS routine → `RET`; library: `GETC`, `OUT`, `PUTS`, `IN`, `PUTSP`, `HALT`
- **Subroutines:** `JSR/JSRR` for your own reusable code; **R7** holds the return address, so save it — and everything else you clobber

Next lecture

- The stack: one clean place to save registers, arguments, and return addresses

Code from today: `echo_traps.asm`, `hello.asm`, `putsp_demo.asm`, `mult_sub.asm`.